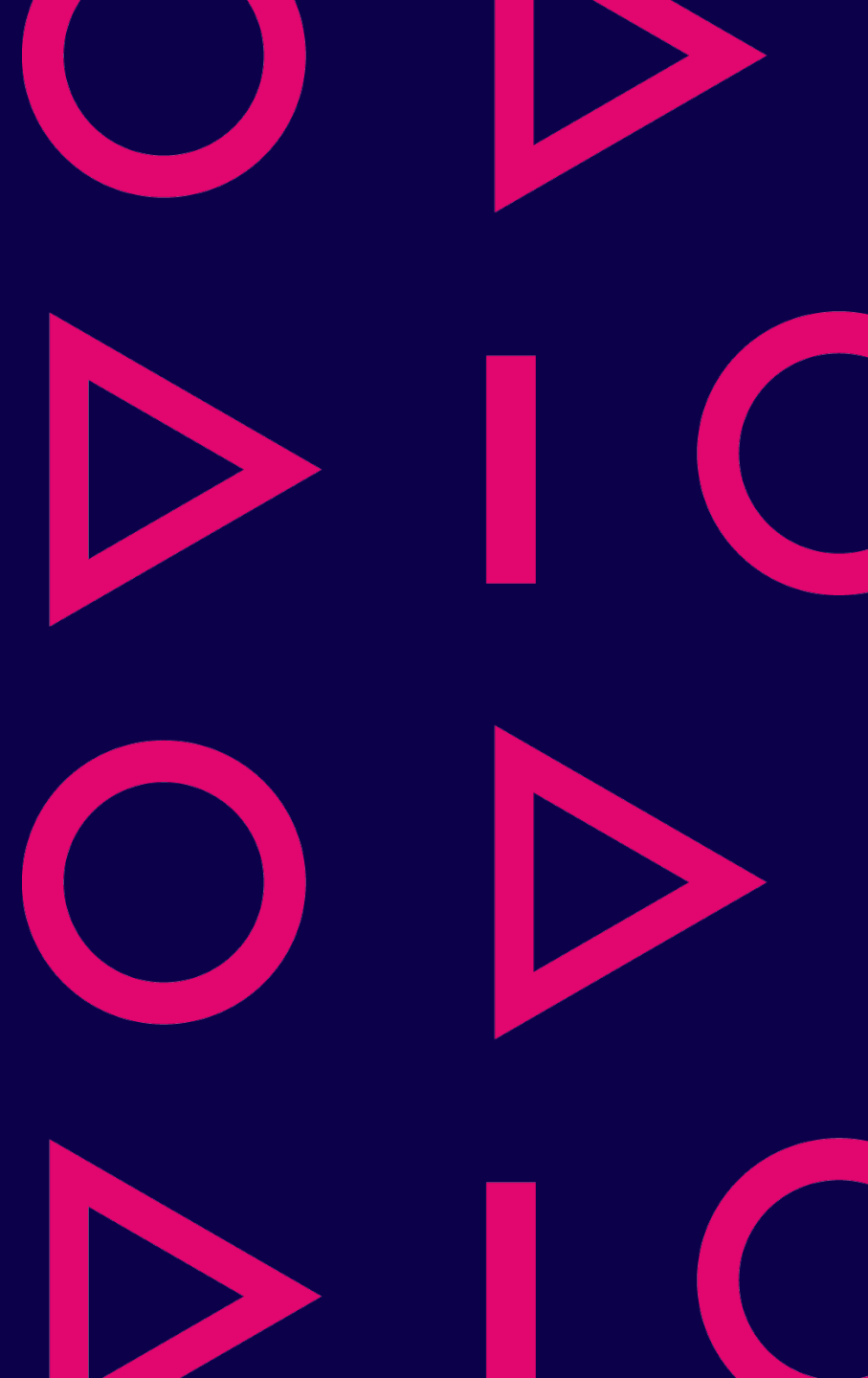


Basics of automation technology

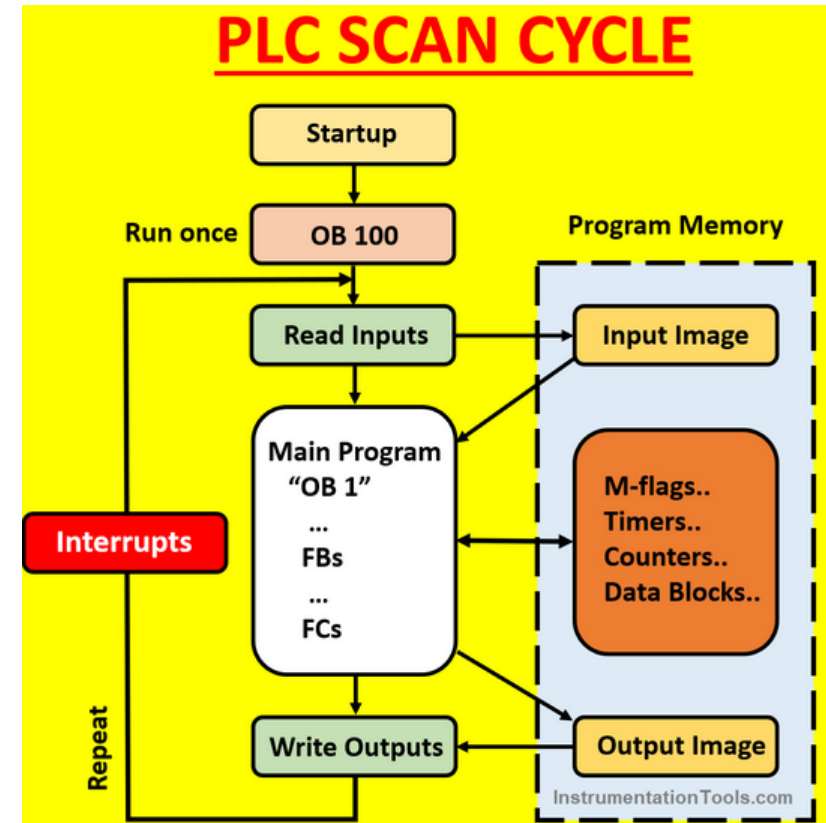
Samppa Alanen

Ref. Teppo Flyktman



The operating cycle of logic

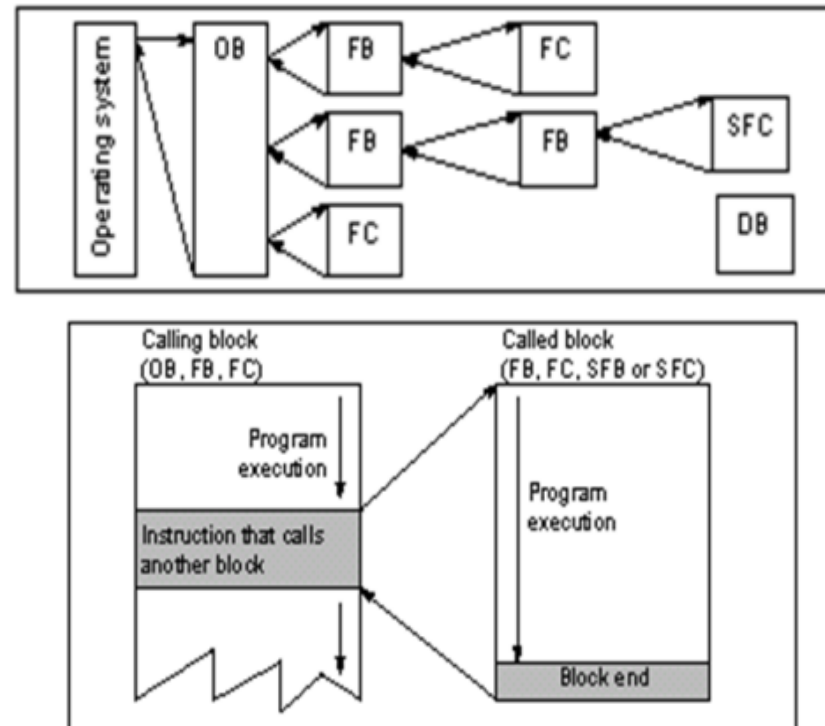
- The program rotation/operating cycle of the logic CPU can be divided into three main stages:
 1. Read the physical inputs and store their states in the “I” memory area (= process image of the inputs = PII = Process Image Inputs).
 2. Run the user program (OB and the subroutines called in it)
 3. Set physical outputs based on the states of bits in the Q - memory area (= process image of the outputs = PIQ = Process Image Outputs)
- When the logic is in RUN mode, the execution of the program cycle is continuous. After point three, the execution starts again from the beginning (read inputs, etc.).
- The cycle time in logics is typically in the order of milliseconds or tens of milliseconds.



Structure and order of execution of the program

- In logic programming, different functions or entities are usually divided into their own subroutines (FC or FB at Siemens), which are called from the common main program (at Siemens OB1).
- In addition to the main program, interruption programs can be used in programming, such as cyclic interrupt OB35, which is executed at intervals of time set in it (e.g. 200ms execution interval). For the duration of the interrupt program, logic interrupts "normal program execution", performs the actions programmed in the interrupt program, and then returns to resume normal program execution from where it was before the interruption. Interrupt programs are not handled or used in this basic course any further

The advantage of modular and clear structured programming is that even large program entities are easier to perceive compared to an unstructured program!



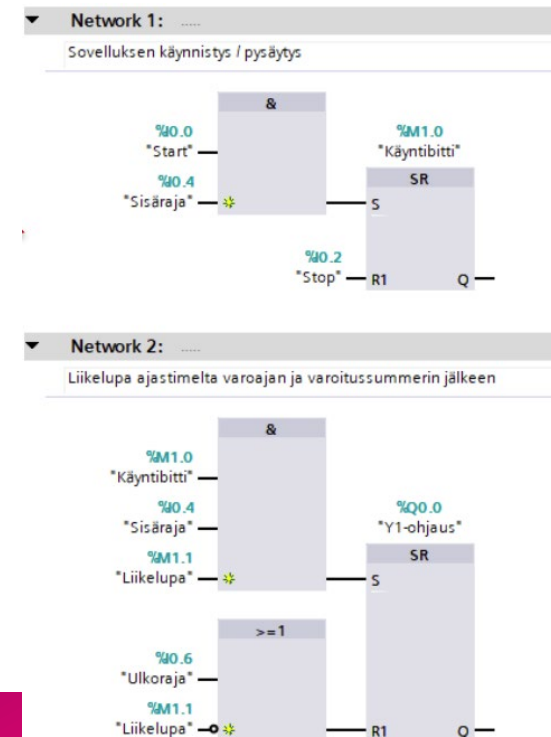
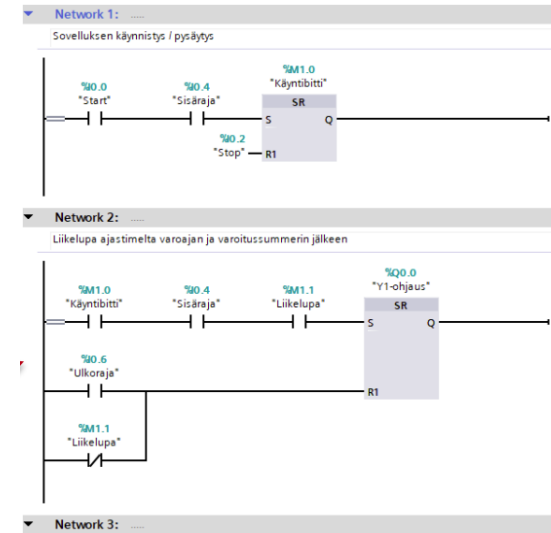
The sequence of execution of the program typically from left to right and from top to bottom.

For a subroutine call, the processor "jumps" to run the subroutine to be called, and then goes back to continue the program forward at the subroutine call.

Programming languages

- Standardin IEC 61131-3 mukaiset kielet:
- Instruction list, IL
 - Siemens Instruction List AWL / STL
- Ladder diagram, LD
 - Siemens Ladder Logic KOP / LAD
- **Function Block Diagram, FBD**
 - Siemens Function Block Diagram FUP/FBD
- Structured Text, ST
 - Siemens Structred Control Language, SCL
- Sequential Function Chart, AS / SFC
 - Siemens Structured Control Language S7-Graph

The available programming languages depend on the logic.

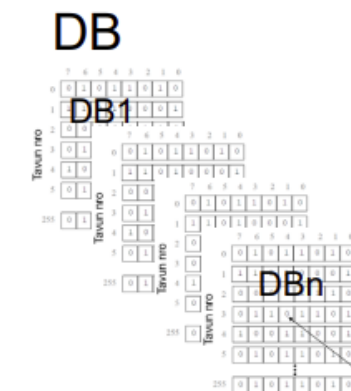
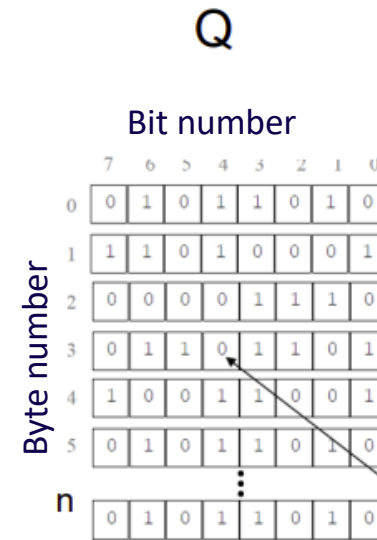
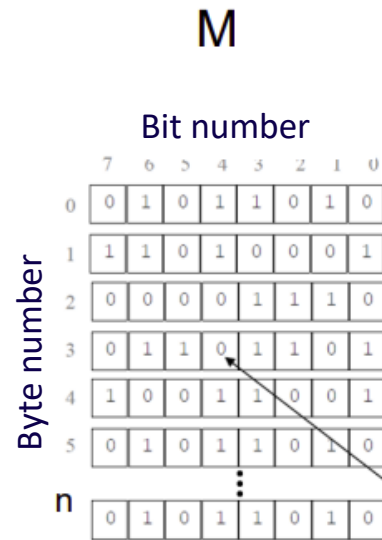
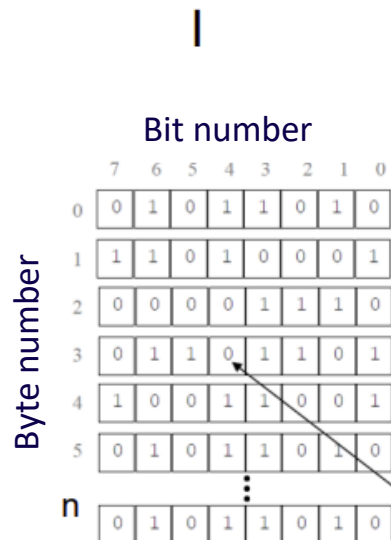


Memory areas in Siemens logic

- I** The states of the physical input signals can be read from the "input image" (PII), memory area **I** - e.g. I0.6
- Q** Physical outputs are placed in the states of the "output image" (PIQ) memory area **Q** - eg. Q0.5
- M** For internal needs of the program ("markers"): memory area **M** – e.g. M1.0
- DB** For in-program needs and memory structuring. In real applications, this range is used more than the M range. (we don't use Data Blocks yet in this course)

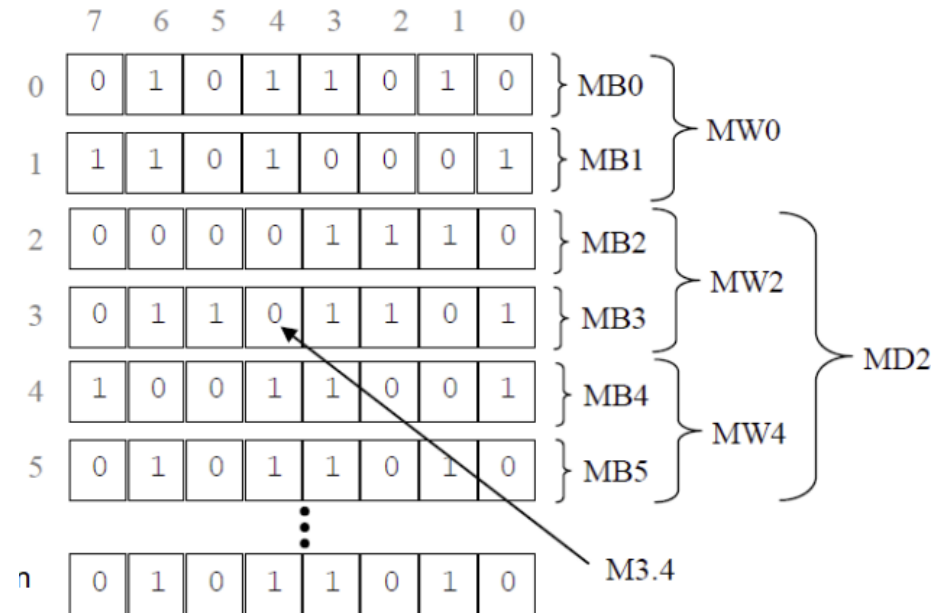
I, Q and M – memory areas are fixed size (size depends on the CPU). Instead, DB must be added by the programmer to the program itself and define the size of the memory inside it, data types, etc.

Memory areas are separate global memory regions. Temp variables can be also used (with caution), but more of these in other courses.



Memory area structure and assignment

- The smallest unit of memory is a bit (as a state of 0 or 1). Eight bits form a byte and two bytes form one word (16bit). Two words form a double word (32bit).



Name	Size	Struct.	e.g. address
Bit	1 bit	0 or 1	M1.2
Byte	8 bit	8bit	MB3
Word	16 bit	2 bytes	MW4
Doubleword	32 bit	4 bytes	MD5

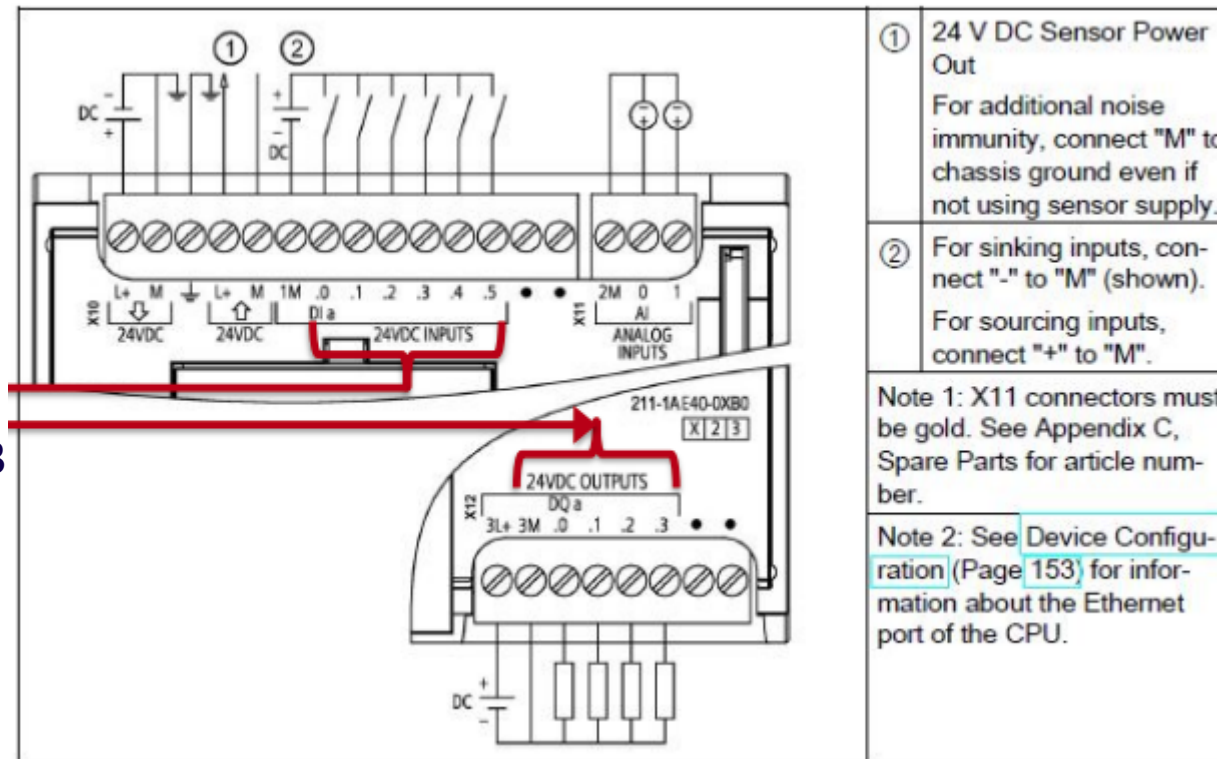
- In the I, Q and M memory areas, the structure of the memory is exactly the same and the memory slots are referred to on the same principle! Only the letter changes
- The first letter of the address indicates the memory area, the second indicates the size of the variable to be assigned, and the number after it is **the byte number from which the memory area/variable begins**. In the case of bits, the bit number is separated by a period after the byte address

I and Q memory

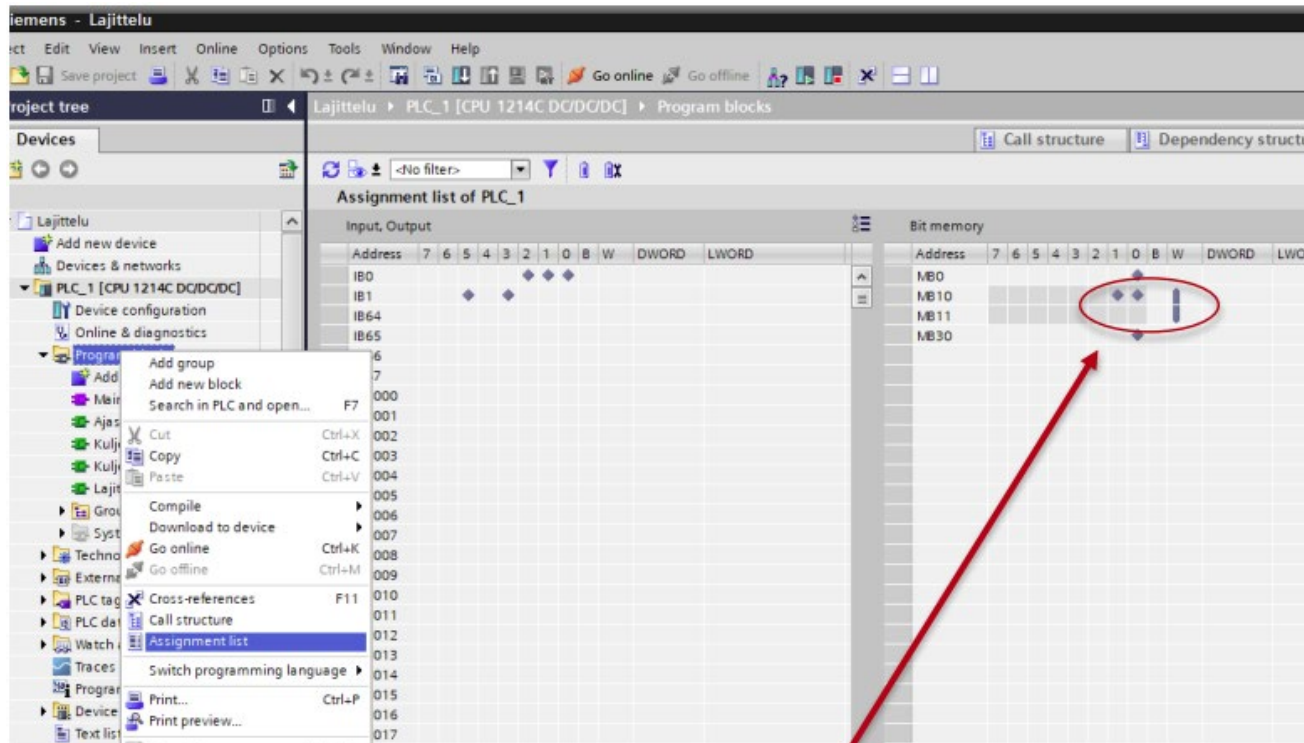
- I = input memory (physical inputs). Addresses connected to physical inputs (connector screws) are defined in the Device Configuration section of the TIA project (→ CPU or IO module → Properties → somewhere below this there is a section referring to inputs and outputs, under which you can specify the memory addresses that connect to the physical input and output connectors)
- Q = Output memory (physical outputs). The connection between the output addresses and the physical outputs can be defined as in inputs.

e.g. I0.0 – I0.5

e.g. Q0.0 – I0.3



Used memory slots



There is a typical overlap here. Two **bits** hit the area reserved by the **word**. The programmer must take care not to use duplicate memory slots!

- The Assignment List displays the memory slots you have used and available. The list opens when you right-click on e.g. Program Blocks folder and select Assignment List
- The list does not say where or how (read or write) memory slots have been used. This information can be found by selecting Cross Reference for the variable.

Datatypes

- **BOOL**
 - The BOOL data type in logic programming is typically one bit in size
 - Its value can be 1 (true) or 0 (False)
 - This is familiar from the connection of logical operations
 - For example, the address of a BOOL-type variable might be: M43.4, I124.0, DB4.DBX4.0 (DataBlock 4, byte 4, bit 0)
- **Bit string**
 - A variable the size of a byte, word, or double word, the content of which is not "interpreted" as any particular type of number or value by default –it is only a variable's worth of bits. For example, an income byte that shows address statuses
- **Integer**
 - Integer does not contain a decimal part
 - Available sizes depend on logic
 - Integer variables beginning with U are uncharacterized, meaning that negative values cannot be stored in them
- **Floating-point**
 - Real is datatype that has decimal part
 - May be positive or negative
 - The REAL data type is two words long, which means that it can be stored, for example, at MD100 (M memory range, Double – i.e. double word and byte number from which the variable begins)
 - If a higher resolution or a wider range of numbers is required, newer logics can use the LREAL data type, which is 64-bit

Binary numbers	S7-300/400	S7-1200	S7-1500	
BOOL	X	X	X	M99.7
Bit strings				
BYTE	X	X	X	MB100
WORD	X	X	X	MW100
DWORD	X	X	X	MD100
LWORD	-	-	X	

Integers	S7-300/400	S7-1200	S7-1500
SINT 8 -bittinen	-	X	X
INT 16 -bittinen	X	X	X
DINT 32 -bittinen	X	X	X
USINT 8 –bittinen	-	X	X
UINT 16 –bittinen	-	X	X
UDINT 32 -bittinen	-	X	X
LINT 64 -bittinen	-	-	X

Floating-point numbers	S7-300/400	S7-1200	S7-1500
REAL	X	X	X
LREAL	-	X	X